

IoT Field Training

First Project



*Dana Natsheh
Cyber Robot
IoT*



1. EXECUTIVE SUMMARY

This report presents the implementation and deployment of a smart sensor and lighting control unit using the Espressif ESP32 microcontroller. Built inside the PlatformIO IDE framework on VS Code, the system integrates an Infrared (IR) proximity sensor and a potentiometer to execute layered digital and analog tracking. The system uses an interlocking logic architecture: an active indicator LED is kept entirely off by default, turning on only when the digital IR sensor detects a nearby object. Once active, its real-time brightness is dynamically adjusted from 0% to 100% using hardware-driven Pulse Width Modulation (PWM) tied directly to the analog position of the potentiometer knob. System diagnostics, including raw sensor counts, calculated input voltage, and actual LED intensity, are piped continuously via an asynchronous UART connection to a monitoring workstation at 115200 baud. All project deliverables have been successfully modularized and pushed into the centralized monorepo structure on GitHub, with collaborator permissions granted for technical evaluation.

2. HARDWARE ARCHITECTURE & PIN MAPPING

The project utilizes an Espressif ESP32 DevKit v1 development board powered via a USB interface. All peripherals operate on a shared 3.3V logic level power rail provided by the board's onboard voltage regulator.

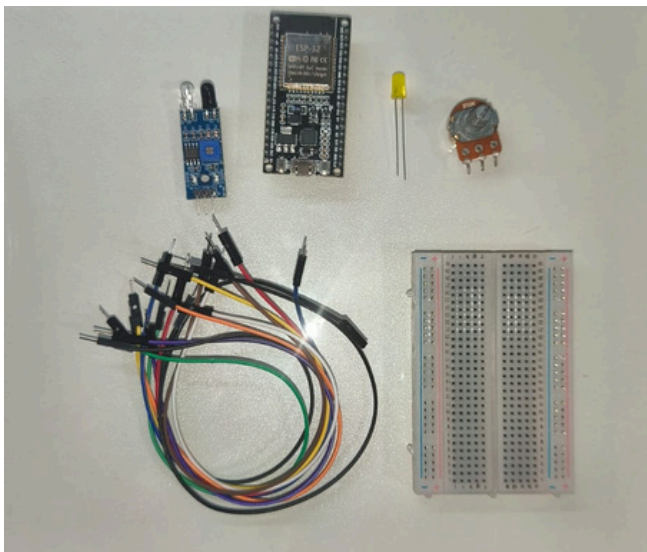
Peripheral	Peripheral Pin	ESP32 GPIO	Electrical Logic / Notes
IR Sensor	VCC	3.3V	Main power rail
IR Sensor	GND	GND	Common ground reference
IR Sensor	OUT	GPIO 25	Digital Input (Active LOW)
Potentiometer	Left Pin	3.3V	Voltage reference high
Potentiometer	Middle Pin	GPIO 34	Analog Input (0V to 3.3V)
Potentiometer	Right Pin	GND	Ground reference
LED Circuit	Anode (+)	GPIO 26	PWM Output
LED Circuit	Cathode (-)	GND	Ground reference



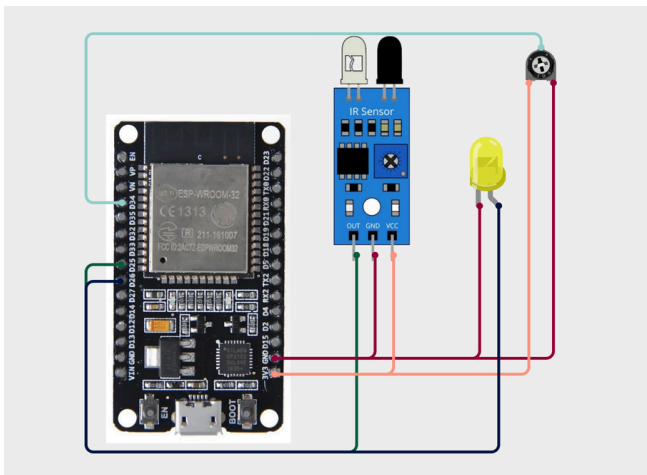


3.SCEMATIC AND COMPONENTS

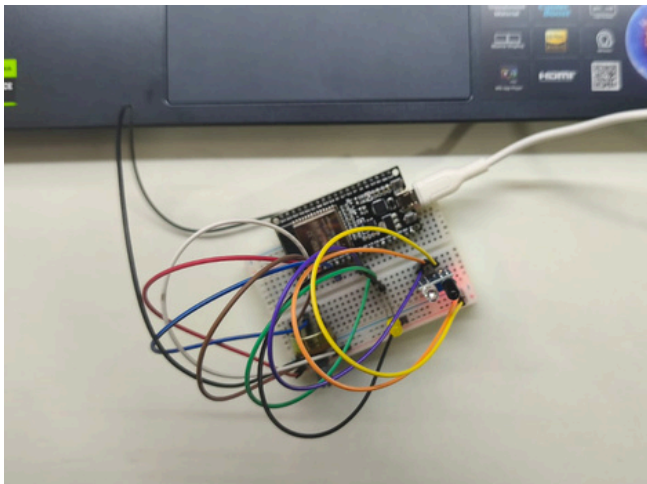
Components:



Scematic:



Assembled circuit:



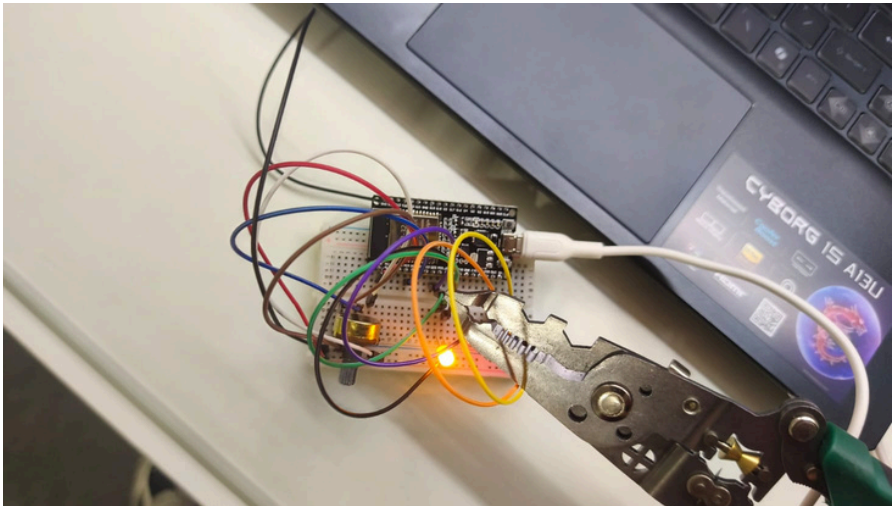


4. TESTED CIRCUIT & SERIAL MONITOR OUTPUT

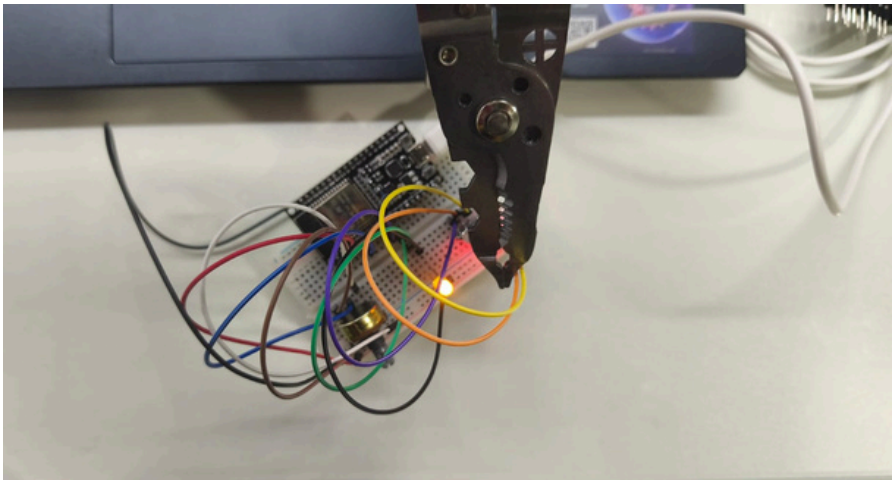
Sample Output:

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	MEMORY	XR10S	SERIAL MONITOR	ESP-IDF
Executing task: C:\Users\User\.platformio\penv\Scripts\platformio.exe device monitor --port COM3								
IR Activated: YES	Potentiometer	Count: 2775	Voltage: 2.24V	Effective LED Intensity: 67.5%				
IR Activated: YES	Potentiometer	Count: 3123	Voltage: 2.52V	Effective LED Intensity: 76.3%				
IR Activated: YES	Potentiometer	Count: 3293	Voltage: 2.65V	Effective LED Intensity: 80.1%				
IR Activated: YES	Potentiometer	Count: 4095	Voltage: 3.30V	Effective LED Intensity: 100.0%				
IR Activated: YES	Potentiometer	Count: 4095	Voltage: 3.30V	Effective LED Intensity: 100.0%				
IR Activated: YES	Potentiometer	Count: 4095	Voltage: 3.30V	Effective LED Intensity: 100.0%				
IR Activated: YES	Potentiometer	Count: 4095	Voltage: 3.30V	Effective LED Intensity: 100.0%				
IR Activated: YES	Potentiometer	Count: 3211	Voltage: 2.59V	Effective LED Intensity: 78.6%				
IR Activated: YES	Potentiometer	Count: 2864	Voltage: 2.31V	Effective LED Intensity: 69.9%				
IR Activated: YES	Potentiometer	Count: 2835	Voltage: 2.28V	Effective LED Intensity: 69.4%				
IR Activated: YES	Potentiometer	Count: 2544	Voltage: 2.05V	Effective LED Intensity: 62.0%				
IR Activated: YES	Potentiometer	Count: 2215	Voltage: 1.78V	Effective LED Intensity: 54.4%				
IR Activated: YES	Potentiometer	Count: 2210	Voltage: 1.78V	Effective LED Intensity: 54.0%				
IR Activated: NO	Potentiometer	Count: 2199	Voltage: 1.77V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 2127	Voltage: 1.71V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1891	Voltage: 1.52V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1535	Voltage: 1.24V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1271	Voltage: 1.02V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1250	Voltage: 1.01V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1223	Voltage: 0.99V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 1042	Voltage: 0.84V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 800	Voltage: 0.64V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 801	Voltage: 0.65V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 798	Voltage: 0.64V	Effective LED Intensity: 0.0%				
IR Activated: NO	Potentiometer	Count: 796	Voltage: 0.64V	Effective LED Intensity: 0.0%				

Serial Monitor Output



LED High Intensity



LED low Intensity





5. HOW THE SYSTEM WORKS

1. **System Gating:** The system treats the digital IR sensor input as an operational master gate. The ESP32 constantly polls the status of the IR sensor and the voltage level of the potentiometer wiper.
2. **Active State Execution:** When the IR sensor reads LOW (Target Active), the conditional logic routes power to the hardware PWM timers. The microcontroller reads the potentiometer wiper ADC register, translates the 12-bit information (0-4095) down to 10-bit resolution (0-1023), and dynamically scales the LED intensity.
3. **Inactive State Interlock:** When the IR sensor reads HIGH (Target Inactive), the firmware overrides all potentiometer positions, setting the PWM output registers directly to 0. This ensures total firmware control over peripheral states based on priority input gates.
4. **Telemetry Stream:** A structured data string formatting the raw ADC counts, scaled voltage levels, calculated LED brightness percentages, and IR trigger states is pushed via UART over the USB bridge at 115200 baud every 250 milliseconds.

6. SOFTWARE IMPLEMENTATION

Main Code:

```
#include <Arduino.h>
```

```
const int irPin = 25;
const int potPin = 34;
const int ledPin = 26;
```

```
const int pwmFreq = 5000;
const int pwmResolution = 10;
const int pwmChannel = 0;
```

```
void setup() {
  Serial.begin(115200);
```

```
  pinMode(irPin, INPUT);
  pinMode(potPin, INPUT);
```

```
  ledcSetup(pwmChannel, pwmFreq, pwmResolution);
  ledcAttachPin(ledPin, pwmChannel);
```

```
}
```

```
void loop() {
  int irState = digitalRead(irPin);
  int pwmValue = 0;
  float brightnessPercent = 0.0;
```

```
  if (irState == LOW) {
    int potValue = analogRead(potPin);
    pwmValue = map(potValue, 0, 4095, 0, 1023);
    ledcWrite(pwmChannel, pwmValue);
    brightnessPercent = ((float)pwmValue / 1023.0) * 100.0;
  }
```





6. SOFTWARE IMPLEMENTATION

```
else {  
  ledcWrite(pwmChannel, 0);  
  pwmValue = 0;  
  brightnessPercent = 0.0;  
}  
  
int currentPotRaw = analogRead(potPin);  
float voltage = (currentPotRaw / 4095.0) * 3.3;  
  
Serial.print("IR Activated: ");  
Serial.print(irState == LOW ? "YES" : "NO");  
Serial.print(" | Potentiometer Count: ");  
Serial.print(currentPotRaw);  
Serial.print(" | Voltage: ");  
Serial.print(voltage, 2);  
Serial.print("V | Effective LED Intensity: ");  
Serial.print(brightnessPercent, 1);  
Serial.println("%");  
  
delay(250);  
}
```

DAILY REPORT: TODAY

- Goal: Write the interlocking controller logic, handle sensor calibration, resolve data stream discrepancies, and organize the GitHub repository structure.
- What I did: Implemented nested conditional checks to lock potentiometer control inside active IR sensor windows. Fixed a serial data corruption issue by adjusting configurations to a uniform 115200 baud rate profile. Reorganized the local directory tree to resolve submodule and directory tracking issues, mapping esp project and project_1 as accessible folders sitting side-by-side.
- Results: The system is operational and stable. The LED stays forced off when the IR path is clear and modulates brightness fluidly based on potentiometer knob translation when blocked. All repository configurations are active online.

